Deep Learning Projects

Data Import → EDA → Preprocessing → Model-building → Evaluation → Saving

Handwritten Digit Classification — Deep Learning (CNN)

3.1 Objective

Classify handwritten digit images (0–9) using a convolutional neural network — a 10-class image classification problem.

3.2 Data Import

```
import kagglehub
path = kagglehub.dataset_download("olafkrastovski/handwritten-digits-0-9")
```

Dataset downloaded directly from Kaggle Hub. Ten sub-folders (one per digit 0–9), each holding several thousand images (2,016–2,241 per class — a well-balanced dataset). A DataFrame of (image_path, label) pairs is built by walking the folders, then shuffled with a fixed random_state.

3.3 Exploratory Data Analysis

- Sample images displayed in a 10×4 grid, four examples per digit class.
- Per-class image counts printed to confirm class balance (all classes within ~10% of each other — no resampling needed).
- Checked native image size of a sample file (140×90 px).

3.4 Preprocessing & Split

- 70/15/15 split by rows: 15,088 train / 3,233 validation / 3,234 test images.

```
data_generator = ImageDataGenerator(
 rescale=1.0/255, rotation_range=40, width_shift_range=0.2,
 height_shift_range=0.2, shear_range=0.2, zoom_range=0.2,
 horizontal_flip=True, fill_mode='nearest')
```

- flow_from_dataframe() used to build train/validation/test generators, resizing every image to 128×128×3, batch size 32, class_mode='categorical'.

3.5 Model Architecture

```
Sequential([
 Input(128,128,3),
 Conv2D(32,3x3,relu), MaxPooling2D,
 Conv2D(64,3x3,relu), MaxPooling2D,
 Conv2D(64,3x3,relu), MaxPooling2D,
 Flatten(), Dense(64,relu), Dense(10,softmax)
])
```

Compiled with Adam optimizer, categorical_crossentropy loss, tracking accuracy. Trained for 20 epochs with the augmented train generator and the (also augmented) validation generator.

3.6 Evaluation

- Training and test accuracy printed via model.evaluate() on the respective generators.
- Accuracy/loss curves plotted for train vs. validation across the 20 epochs.
- A single external image (three.jpg) is loaded, resized, normalized, and classified — predicted class printed via argmax on the softmax output.

3.7 Model Saving

No model-saving cell is present in this notebook — the workflow stops after the single-image prediction. The outline calls for saving to .h5/.keras and reloading it, but that step was not implemented here.

3.8 Pros

- Clean, reproducible data-loading pipeline straight from Kaggle Hub with no manual file wrangling.
- Class balance is explicitly checked before training, which matters for a 10-class problem — an unbalanced dataset here would bias the model toward majority digits.
- A simple, appropriately-sized 3-block CNN is a sensible architecture for 128×128 digit images — not over-engineered for the task.
- The end-to-end loop (train → evaluate → external image prediction) proves the model generalizes beyond the training set images, which is a good sanity check.

3.9 Cons, Challenges & How to Handle Them

- **Augmentation leaks into validation/test:** the same heavily-augmented data_generator (rotation, shift, shear, zoom, flip) is reused to build the validation and test generators, not just the training one. Augmenting evaluation data makes accuracy/loss metrics noisy and non-reproducible from run to run, and isn't standard practice. Fix: create a second ImageDataGenerator(rescale=1./255) with no augmentation, and use it for validation_generator and test_generator, keeping the augmented one only for train_generator.
- **Horizontal flip is inappropriate for digits:** horizontal_flip=True mirrors digit strokes left-right, which can turn a valid digit into a different or invalid shape (e.g. a flipped '2' or '3' no longer looks like a real handwritten digit). Fix: disable horizontal_flip (and consider disabling vertical operations too) for digit-style data; keep only mild rotation/shift/zoom.
- **Evaluation metrics are incomplete:** only accuracy/loss are reported; the plan calls for precision, recall, F1-score, and a confusion matrix, which are especially useful for spotting which digit pairs (e.g. 4 vs 9, 3 vs 8) the model confuses. Fix: collect predictions over the full test_generator, then use sklearn.metrics.classification_report and confusion_matrix (visualized with a heatmap).
- **Only one model configuration trained:** the plan mentions comparing multiple models/hyperparameter settings, but only a single CNN architecture is trained once. Fix: try at least one variant (e.g. added dropout, different learning rate, or a slightly deeper network) and compare validation accuracy before picking a final model.

Unnecessary 3-channel input: digit images are inherently single-channel (grayscale strokes on a background), but the generator loads them as 3-channel RGB, tripling the input size and first-layer parameter count for no expected accuracy gain. Fix: set color_mode='grayscale' in flow_from_dataframe and adjust the Input shape to (128,128,1).